

(Optional) Colab Setup

If you aren't using Colab, you can delete the following code cell. This is just to help students with mounting to Google Drive to access the other .py files and downloading the data, which is a little trickier on Colab than on your local machine using Jupyter.

```
In [ ]: # you will be prompted with a window asking to grant permissions
from google.colab import drive
drive.mount("/content/drive")
```

```
In [ ]: # fill in the path in your Google Drive in the string below. Note: do not escape
import os
datadir = "/content/assignment1"
if not os.path.exists(datadir):
    !ln -s "/content/drive/My Drive/YOUR PATH HERE/assignment1/" $datadir
os.chdir(datadir)
!pwd
```

```
In [ ]: # downloading Fashion-MNIST
import os
os.chdir(os.path.join(datadir, "fashion-mnist/"))
!chmod +x ./get_data.sh
!./get_data.sh
os.chdir(datadir)
```

Imports

```
In [1]: import random
import numpy as np
from data_process import get_FASHION_data, get_RICE_data
from scipy.spatial import distance
from models import Perceptron, SVM, Softmax, Logistic
from kaggle_submission import output_submission_csv
%matplotlib inline

# For auto-reloading external modules
# See http://stackoverflow.com/questions/1907993/autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2
```

Loading Fashion-MNIST

In the following cells we determine the number of images for each split and load the images.

TRAIN_IMAGES + VAL_IMAGES = (0, 60000] , TEST_IMAGES = 10000

```
In [2]: # You can change these numbers for experimentation
# For submission we will use the default values
```

```
TRAIN_IMAGES = 50000
VAL_IMAGES = 10000
normalize = True
```

```
In [3]: data = get_FASHION_data(TRAIN_IMAGES, VAL_IMAGES, normalize=normalize)
X_train_fashion, y_train_fashion = data['X_train'], data['y_train']
X_val_fashion, y_val_fashion = data['X_val'], data['y_val']
X_test_fashion, y_test_fashion = data['X_test'], data['y_test']
n_class_fashion = len(np.unique(y_test_fashion))
```

Loading Rice

```
In [4]: # Loads train / test / val splits of 80%, 20%, 20%
data = get_RICE_data()
X_train_RICE, y_train_RICE = data['X_train'], data['y_train']
X_val_RICE, y_val_RICE = data['X_val'], data['y_val']
X_test_RICE, y_test_RICE = data['X_test'], data['y_test']
n_class_RICE = len(np.unique(y_test_RICE))

print("Number of train samples: ", X_train_RICE.shape[0])
print("Number of val samples: ", X_val_RICE.shape[0])
print("Number of test samples: ", X_test_RICE.shape[0])
```

```
Number of train samples: 10911
Number of val samples: 3637
Number of test samples: 3637
```

Get Accuracy

This function computes how well your model performs using accuracy as a metric.

```
In [5]: def get_acc(pred, y_test):
        return np.sum(y_test == pred) / len(y_test) * 100
```

Perceptron

Perceptron has 2 hyperparameters, "learning rate" and "number of epochs" that you can experiment with:

Hyperparameter 1: Learning rate

The learning rate controls how much we change the current weights of the classifier during each update. We set it at a default value of 0.5, but you should experiment with different values. Here is a guide to help you find a right learning rate:

- Try values ranging from 0.0005 to 5.0 to see the impact on model accuracy.
- If the loss oscillates a lot or the loss diverges, the learning rate may be too high. Try decreasing it by a factor of 10 (e.g. from 0.5 to 0.05).
- If the loss decreases very slowly, the learning rate may be too low and the training can take a long time. Try increasing it by a factor of 10.

- You can also try adding a learning rate decay to slowly reduce the learning rate over each training epoch. For example, multiply the learning rate by 0.95 after each epoch.
- Plot training and validation accuracy over epochs for different learning rates. This will help you visualize the impact of the learning rate.
- [Here](#) is a detailed guide to learning rate.

Hyperparameter 2: Number of Epochs

- An epoch is a complete iterative pass over the training dataset. **During an epoch we predict a label using the classifier and then update the weights of the classifier according to the perceptron update rule for each sample in the training set.**
- By observing the training and validation accuracy graphs, you can determine if the model has converged or not. Try increasing the number of epochs if the model is not converging or has low accuracy.
- You should try different values for the number of training epochs and report your results.

In this section, you will implement the Perceptron classifier in the **models/perceptron.py** script.

The following code:

- Creates an instance of the Perceptron classifier class
- An instance of the Perceptron class is trained on the training data using its train method.
- We use the predict method to find the training accuracy as well as the testing accuracy

Train Perceptron on Fashion-MNIST

```
In [6]: lr = 0.0005
n_epochs = 20
percept_fashion = Perceptron(n_class_fashion, lr, n_epochs)

# - Refer to hints in the train method of the Perceptron class in "models/percep
percept_fashion.train(X_train_fashion, y_train_fashion)
```

```
In [7]: pred_percept = percept_fashion.predict(X_train_fashion)
print('The training accuracy is given by: %f' % (get_acc(pred_percept, y_train_f
```

The training accuracy is given by: 87.454000

Validate Perceptron on Fashion-MNIST

```
In [8]: pred_percept = percept_fashion.predict(X_val_fashion)
print('The validation accuracy is given by: %f' % (get_acc(pred_percept, y_val_f
```

The validation accuracy is given by: 82.470000

Test Perceptron on Fashion-MNIST

```
In [9]: pred_percept = percept_fashion.predict(X_test_fashion)
print('The testing accuracy is given by: %f' % (get_acc(pred_percept, y_test_fas
```

The testing accuracy is given by: 81.900000

Perceptron_Fashion-MNIST Kaggle Submission

Once you are satisfied with your solution and test accuracy, output a file to submit your test set predictions to the Kaggle for Assignment 1 Fashion-MNIST. Use the following code to do so:

```
In [10]: output_submission_csv('kaggle/perceptron_submission_fashion.csv', percept_fashio
```

Train Perceptron on Rice

```
In [11]: lr = 0.05
n_epochs = 50

percept_RICE = Perceptron(n_class_RICE, lr, n_epochs)
percept_RICE.train(X_train_RICE, y_train_RICE)
```

```
In [12]: pred_percept = percept_RICE.predict(X_train_RICE)
print('The training accuracy is given by: %f' % (get_acc(pred_percept, y_train_R
```

The training accuracy is given by: 99.871689

Validate Perceptron on Rice

```
In [13]: pred_percept = percept_RICE.predict(X_val_RICE)
print('The validation accuracy is given by: %f' % (get_acc(pred_percept, y_val_R
```

The validation accuracy is given by: 99.807534

Test Perceptron on Rice

```
In [14]: pred_percept = percept_RICE.predict(X_test_RICE)
print('The testing accuracy is given by: %f' % (get_acc(pred_percept, y_test_RIC
```

The testing accuracy is given by: 99.780038

Support Vector Machines (with SGD)

Next, you will implement a "soft margin" SVM. In this formulation you will maximize the margin between positive and negative training examples and penalize margin violations using a hinge loss.

We will optimize the SVM loss using SGD. This means you must compute the loss function with respect to model weights. You will use this gradient to update the model weights.

SVM optimized with SGD has 3 hyperparameters that you can experiment with:

- **Learning rate** - similar to as defined above in Perceptron, this parameter scales by how much the weights are changed according to the calculated gradient update.
- **Epochs** - similar to as defined above in Perceptron.
- **Regularization constant** - Hyperparameter to determine the strength of regularization. In this case it is a coefficient on the term which maximizes the margin. You could try different values. The default value is set to 0.05.

You will implement the SVM using SGD in the **models/svm.py**

The following code:

- Creates an instance of the SVM classifier class
- The train function of the SVM class is trained on the training data
- We use the predict function to find the training accuracy as well as the testing accuracy

Train SVM on Fashion-MNIST

```
In [15]: # Adjust the following three hyperparameters to achieve the best accuracy:
lr = 0.01
n_epochs = 75
reg_const = 0.5

svm_fashion = SVM(n_class_fashion, lr, n_epochs, reg_const)

# Refer to hints in the calculate_gradient and train methods of the SVM class in
svm_fashion.train(X_train_fashion, y_train_fashion)
```

```
In [16]: pred_svm = svm_fashion.predict(X_train_fashion)
print('The training accuracy is given by: %f' % (get_acc(pred_svm, y_train_fashi
```

The training accuracy is given by: 84.590000

Validate SVM on Fashion-MNIST

```
In [17]: pred_svm = svm_fashion.predict(X_val_fashion)
print('The validation accuracy is given by: %f' % (get_acc(pred_svm, y_val_fashi
```

The validation accuracy is given by: 82.980000

Test SVM on Fashion-MNIST

```
In [18]: pred_svm = svm_fashion.predict(X_test_fashion)
print('The testing accuracy is given by: %f' % (get_acc(pred_svm, y_test_fashion
```

The testing accuracy is given by: 82.440000

SVM_Fashion-MNIST Kaggle Submission

Once you are satisfied with your solution and test accuracy output a file to submit your test set predictions to the Kaggle for Assignment 1 Fashion-MNIST. Use the following code to do so:

```
In [19]: output_submission_csv('kaggle/svm_submission_fashion.csv', svm_fashion.predict(X
```

Draw testing accuracy with different learning rate

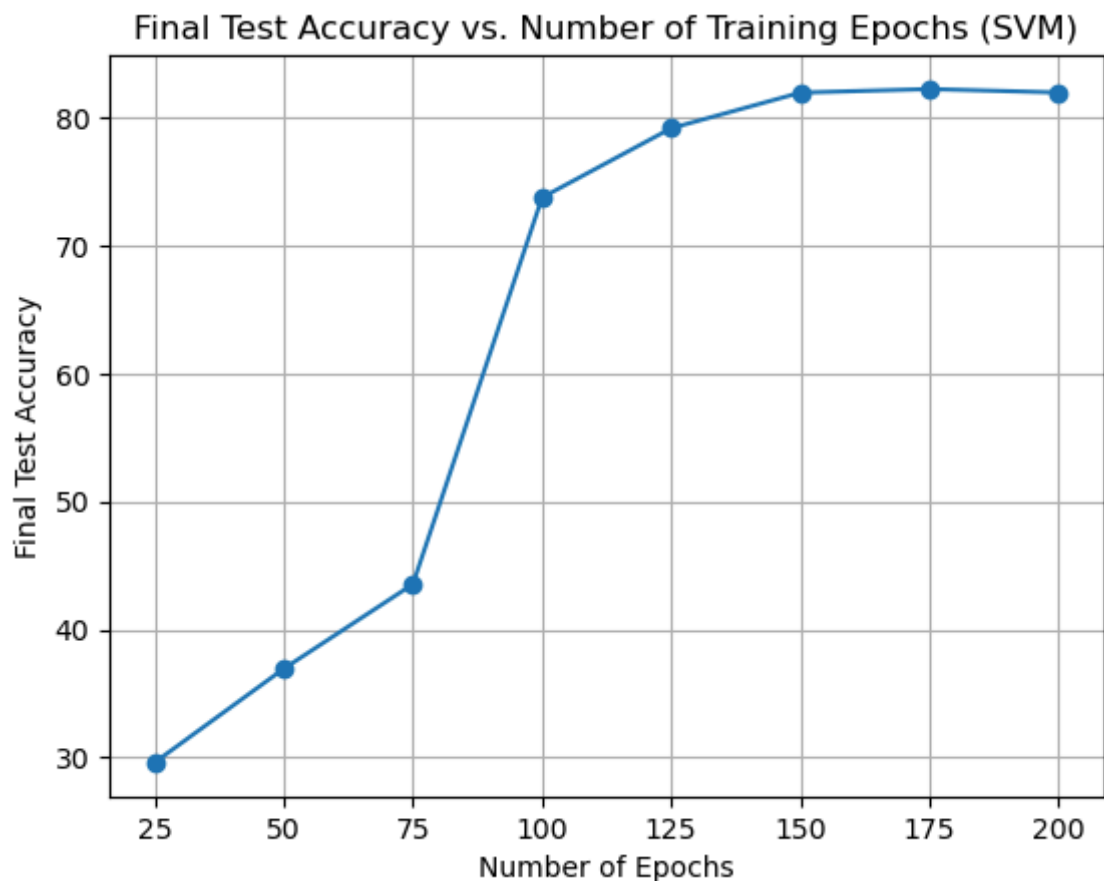
```
In [20]: import matplotlib.pyplot as plt

lr_list = [0.01, 0.05, 0.5, 5]
epochs_list = [25, 50, 75, 100, 125, 150, 175, 200]
reg_const = 0.5
final_test_accuracies = []

for epoch in epochs_list:
    svm_fashion = SVM(n_class_fashion, lr_list[3], epoch, reg_const)
    svm_fashion.train(X_train_fashion, y_train_fashion)

    pred_svm = svm_fashion.predict(X_test_fashion)
    test_accuracy = get_acc(pred_svm, y_test_fashion)
    final_test_accuracies.append(test_accuracy)

plt.figure()
plt.plot(epochs_list, final_test_accuracies, marker='o', linestyle='--')
plt.xlabel("Number of Epochs")
plt.ylabel("Final Test Accuracy")
plt.title("Final Test Accuracy vs. Number of Training Epochs (SVM)")
plt.grid(True)
plt.show()
```



Train SVM on Rice

```
In [21]: # Adjust the following three hyperparameters to achieve the best accuracy:
lr = 0.5
n_epochs = 50
reg_const = 1.0
```

```
svm_RICE = SVM(n_class_RICE, lr, n_epochs, reg_const)
svm_RICE.train(X_train_RICE, y_train_RICE)
```

```
In [22]: pred_svm = svm_RICE.predict(X_train_RICE)
print('The training accuracy is given by: %f' % (get_acc(pred_svm, y_train_RICE))
```

The training accuracy is given by: 97.103840

Validate SVM on Rice

```
In [23]: pred_svm = svm_RICE.predict(X_val_RICE)
print('The validation accuracy is given by: %f' % (get_acc(pred_svm, y_val_RICE))
```

The validation accuracy is given by: 97.195491

Test SVM on Rice

```
In [24]: pred_svm = svm_RICE.predict(X_test_RICE)
print('The testing accuracy is given by: %f' % (get_acc(pred_svm, y_test_RICE)))
```

The testing accuracy is given by: 97.167996

Softmax Classifier (with SGD)

Next, you will train a Softmax classifier. This classifier consists of a linear function of the input data followed by a softmax function which outputs a vector of dimension C (number of classes) for each data point. Each entry of the softmax output vector corresponds to a confidence in one of the C classes, and like a probability distribution, the entries of the output vector sum to 1. We use a cross-entropy loss on this softmax output to train the model.

Check the following link as an additional resource on softmax classification:

<http://cs231n.github.io/linear-classify/#softmax>

Once again we will train the classifier with SGD. This means you need to compute the gradients of the softmax cross-entropy loss function according to the weights and update the weights using this gradient. Check the following link to help with implementing the gradient updates: <https://deeptnotes.io/softmax-crossentropy>

The softmax classifier has 3 hyperparameters that you can experiment with:

- **Learning rate** - As above, this controls how much the model weights are updated with respect to their gradient.

- **Number of Epochs** - As described for perceptron.
- **Regularization constant** - Hyperparameter to determine the strength of regularization. In this case, we minimize the L2 norm of the model weights as regularization, so the regularization constant is a coefficient on the L2 norm in the combined cross-entropy and regularization objective.

You will implement a softmax classifier using SGD in the **models/softmax.py**

The following code:

- Creates an instance of the Softmax classifier class
- The train function of the Softmax class is trained on the training data
- We use the predict function to find the training accuracy as well as the testing accuracy

Train Softmax on Fashion-MNIST

```
In [25]: lr = 0.5
n_epochs = 50
reg_const = 0.0005

softmax_fashion = Softmax(n_class_fashion, lr, n_epochs, reg_const)

# Refer to hints in the calculate_gradient and train methods of the Softmax class
softmax_fashion.train(X_train_fashion, y_train_fashion)
```

```
In [26]: pred_softmax = softmax_fashion.predict(X_train_fashion)
print('The training accuracy is given by: %f' % (get_acc(pred_softmax, y_train_fashion)))
```

The training accuracy is given by: 86.894000

Validate Softmax on Fashion-MNIST

```
In [27]: pred_softmax = softmax_fashion.predict(X_val_fashion)
print('The validation accuracy is given by: %f' % (get_acc(pred_softmax, y_val_fashion)))
```

The validation accuracy is given by: 85.590000

Testing Softmax on Fashion-MNIST

```
In [28]: pred_softmax = softmax_fashion.predict(X_test_fashion)
print('The testing accuracy is given by: %f' % (get_acc(pred_softmax, y_test_fashion)))
```

The testing accuracy is given by: 84.380000

Softmax_Fashion-MNIST Kaggle Submission

Once you are satisfied with your solution and test accuracy output a file to submit your test set predictions to the Kaggle for Assignment 1 Fashion-MNIST. Use the following code to do so:

```
In [29]: output_submission_csv('kaggle/softmax_submission_fashion.csv', softmax_fashion.predict(X_test_fashion))
```

Train Softmax on Rice

```
In [30]: lr = 0.5
         n_epochs = 100
         reg_const = 0.05

         softmax_RICE = Softmax(n_class_RICE, lr, n_epochs, reg_const)
         softmax_RICE.train(X_train_RICE, y_train_RICE)
```

```
In [31]: pred_softmax = softmax_RICE.predict(X_train_RICE)
         print('The training accuracy is given by: %f' % (get_acc(pred_softmax, y_train_RICE)))
```

The training accuracy is given by: 99.880854

Validate Softmax on Rice

```
In [32]: pred_softmax = softmax_RICE.predict(X_val_RICE)
         print('The validation accuracy is given by: %f' % (get_acc(pred_softmax, y_val_RICE)))
```

The validation accuracy is given by: 99.752543

Testing Softmax on Rice

```
In [33]: pred_softmax = softmax_RICE.predict(X_test_RICE)
         print('The testing accuracy is given by: %f' % (get_acc(pred_softmax, y_test_RICE)))
```

The testing accuracy is given by: 99.835029

Logistic Classifier

The Logistic Classifier has 2 hyperparameters that you can experiment with:

- **Learning rate** - similar to as defined above in Perceptron, this parameter scales by how much the weights are changed according to the calculated gradient update.
- **Number of Epochs** - As described for perceptron.
- **Threshold** - The decision boundary of the classifier.

You will implement the Logistic Classifier in the **models/logistic.py**

The following code:

- Creates an instance of the Logistic classifier class
- The train function of the Logistic class is trained on the training data
- We use the predict function to find the training accuracy as well as the testing accuracy

Training Logistic Classifier

```
In [34]: learning_rate = 5
         n_epochs = 150
```

```
threshold = 0.5

lr = Logistic(learning_rate, n_epochs, threshold)

# Refer to hints in the sigmoid and train methods of the Logistic class in "mode
lr.train(X_train_RICE, y_train_RICE)
```

```
In [35]: pred_lr = lr.predict(X_train_RICE)
print('The training accuracy is given by: %f' % (get_acc(pred_lr, y_train_RICE)))
```

The training accuracy is given by: 99.972505

Validate Logistic Classifier

```
In [36]: pred_lr = lr.predict(X_val_RICE)
print('The validation accuracy is given by: %f' % (get_acc(pred_lr, y_val_RICE)))
```

The validation accuracy is given by: 99.917514

Test Logistic Classifier

```
In [37]: pred_lr = lr.predict(X_test_RICE)
print('The testing accuracy is given by: %f' % (get_acc(pred_lr, y_test_RICE)))
```

The testing accuracy is given by: 99.972505

In []: